



# Introduction to Computer Science

## Unit 7: Coordinates

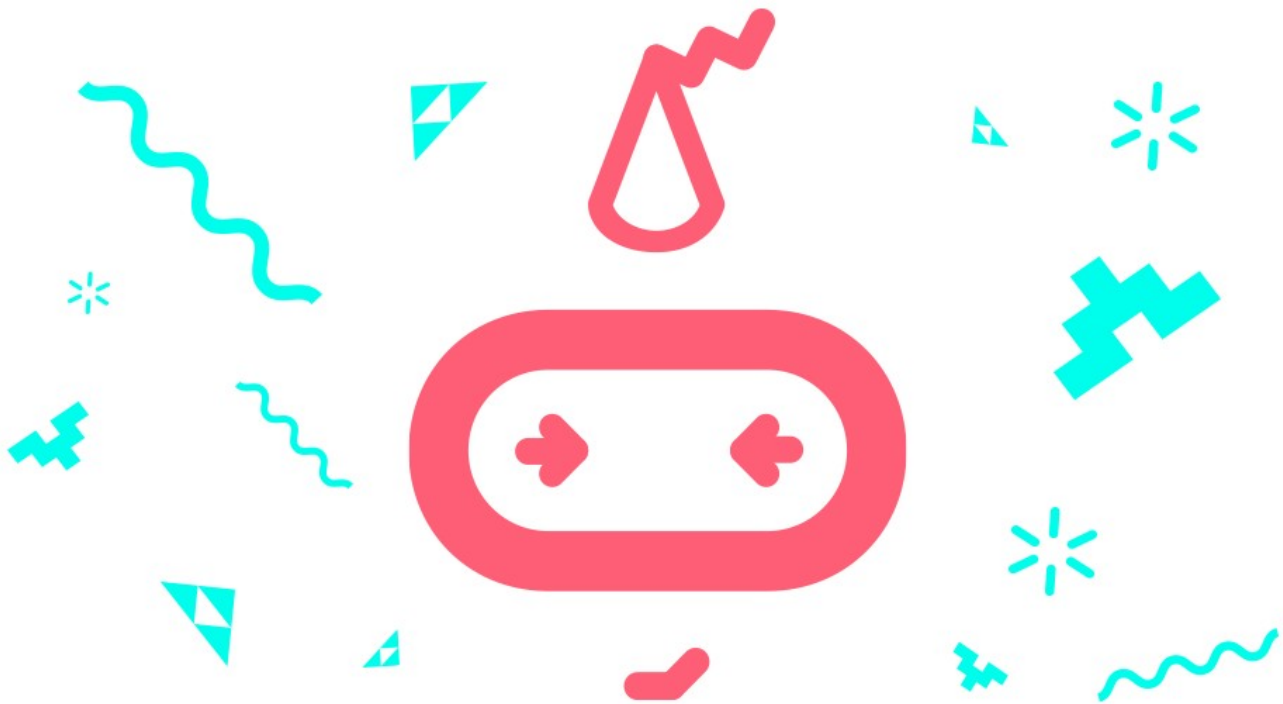
Student workbook  
[makecode.microbit.org](https://makecode.microbit.org)



Table of Contents

---

|                                               |    |
|-----------------------------------------------|----|
| Overview.....                                 | 3  |
| Unit summary.....                             | 3  |
| Learning goals.....                           | 3  |
| Lesson A: Understanding coordinates.....      | 4  |
| Lesson B: Coding animations and patterns..... | 8  |
| Lesson C: Screensaver or game project.....    | 21 |
| Glossary of key terms.....                    | 28 |



# Overview

---

## Unit summary

This unit focuses on the use of coordinates to store data or the results of mathematical operations. You are probably familiar with the concept of coordinates from your math classes. In this unit, you'll first learn the similarities and differences with coordinates on the micro:bit LED grid. Then, in the unplugged activity, you will gain familiarity with the 5x5 grid of coordinates by playing a variation of the game Battleship. In the coding activities, you will practice programming for the LEDs of the micro:bit screen using coordinates and the basic game blocks of MakeCode. In the project, you will create a game or screensaver using the LEDs of the micro:bit in an integral way.

## Learning goals

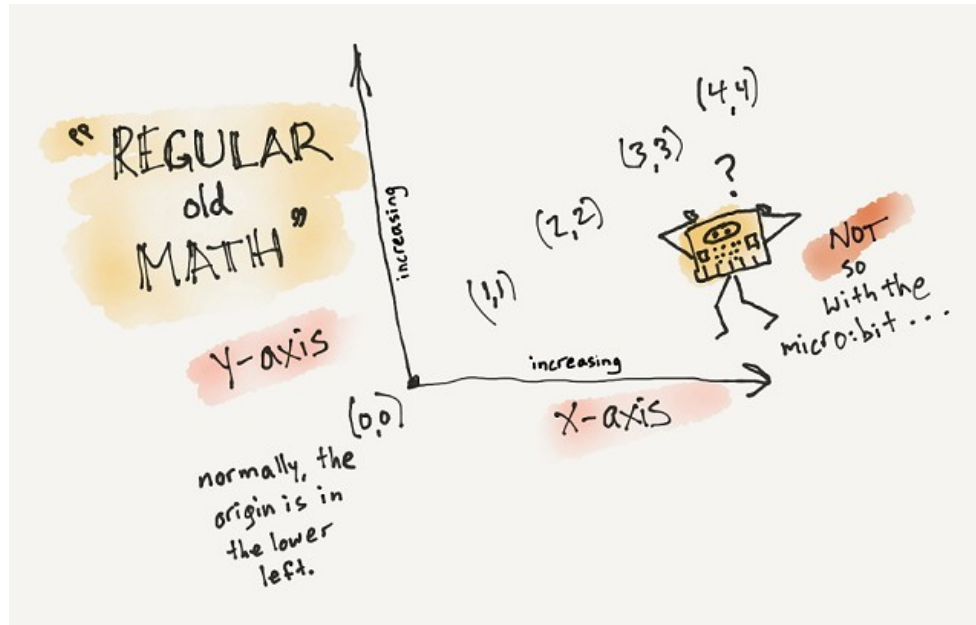
During this unit, you will:

- Understand that the 5x5 grid of LEDs on the micro:bit represents a coordinate grid with the origin (0,0) in the top left corner.
- Understand that the values of the x-coordinates range from 0 through 4 and increase from left to right.
- Understand that the values of the y-coordinates range from 0 through 4 and increase from top to bottom.
- Learn how to refer to an individual LED by its x- and y-coordinates.
- Learn how to plot (turn on) and unplot (turn off) individual LEDs and how to toggle between these two states.
- Learn how to check the current on/off status of an individual LED and check and set the brightness level.
- Apply the above knowledge and skills to create a unique program that uses coordinates as an integral part of the project.

# Lesson A: Understanding coordinates

## Overview: Coordinates

Because of math class, you may already be familiar with coordinate grids and mapping x- and y-coordinates on a plane.



## Axes

The basic coordinate grid taught in math class has two axes:

- an x-axis which runs horizontally (left to right), and
- a y-axis which runs vertically (up to down).

## Origin

These two axes meet at a point called the origin, where both the x- and the y-values are zero. On this basic coordinate grid, the origin is in the lower left corner of the grid and has the coordinates (0,0).

## Coordinate pair

The first value in a coordinate pair is the x-value, and the second value in a coordinate pair is the y-value, e.g., (x,y).

Hint: A simple way to remember which value comes first is to remember their order in the alphabet. The letter x comes before the letter y in the alphabet, and the x-coordinate comes before the y-coordinate in a coordinate pair.

## Coordinate value changes

On a basic coordinate grid:

- The value of the x-coordinate increases from left to right and is a measure of how many units a point is horizontally from the origin.
- The value of the y-coordinate increases from bottom to top and is a measure of how many units a point is vertically from the origin.

## Coordinate grid and JavaScript and the micro:bit

The 5 x 5 grid of LEDs on the micro:bit represents a coordinate grid with a horizontal x-axis and a vertical y-axis. It has an origin and you can refer to the position of the LEDs with coordinate pairs. There are similarities and differences between the micro:bit coordinate grid and basic coordinate grids in math class.



Similar to coordinate grids in math

- The values of the x-coordinates range from 0 through 4 and increase from left to right just as they do in the coordinate grids used in math class.

Different from coordinate grids in math

It is important, however, that you understand the two major differences between the micro:bit LED grid and the coordinate grid that you are used to using in math class:

- The origin (0,0) is in the *top left* corner.
- The values of the y-coordinates range from 0 through 4 and increase from top to bottom.

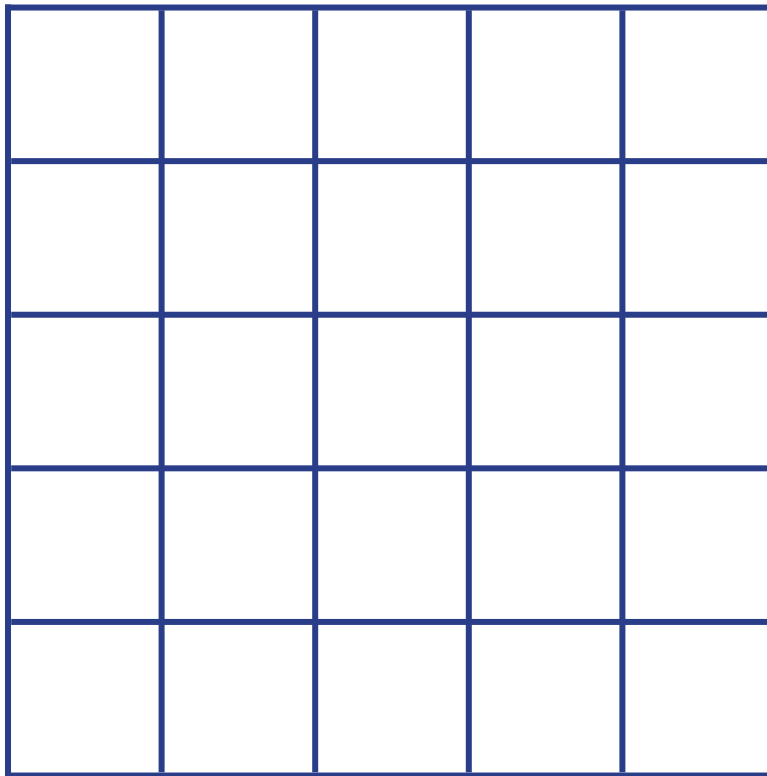
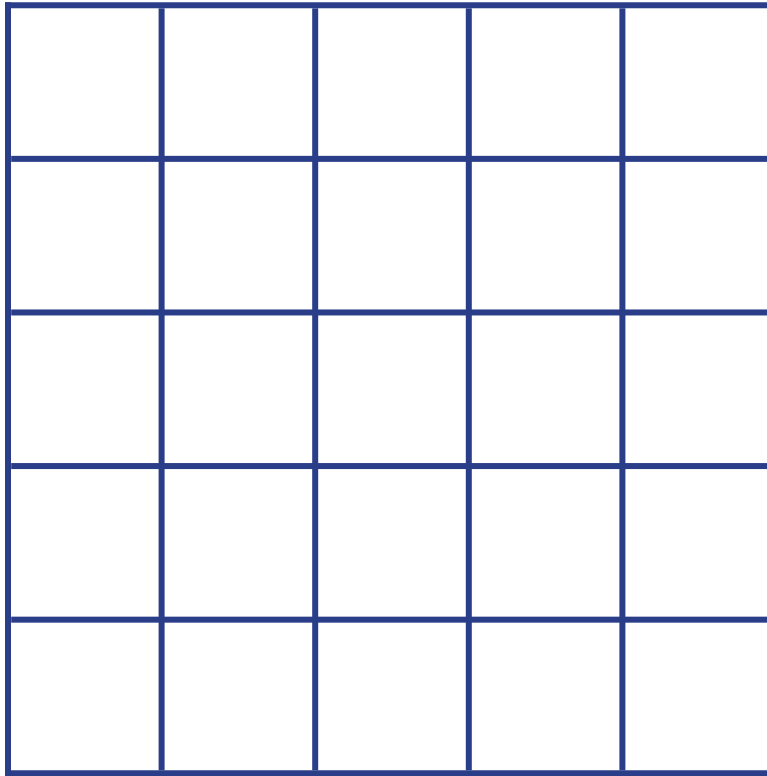
## René Descartes



Image: [Wikipedia Commons](#)

René Descartes (1596-1650) was a French philosopher and mathematician who developed the coordinate system we use today. A story goes that while lying in bed, he noticed a fly on the ceiling. In wondering how he could describe the fly's exact position on the ceiling, he decided to use a corner of the ceiling as a reference point and then describe the fly's position as a measure of how far away from the reference point one would need to travel horizontally and then vertically to reach the fly. His coordinate system proved useful in many ways including creating an important link between the studies of algebra and geometry. Geometric shapes could now be described by points on a coordinate plane.

## Battleship grids



# Lesson B: Coding animations and patterns

---

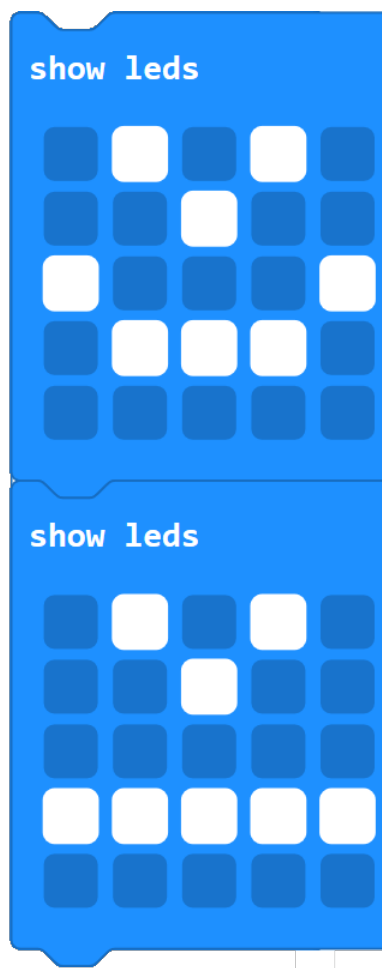
## Coding activity 1: Smile animation

A short exercise in plotting and toggling LEDs to create a simple animation. Though you can use the 'show leds' block for images and animation, there is another way to tell the micro:bit what LEDs to turn on and off using coordinates.

Create a smiling and non-smiling face with 'show leds' blocks

You can still use the 'show leds' block to plan which LED coordinates to turn on.

1. In Microsoft MakeCode, start a new project and name it something like 'smile animation'.
2. From the Basic Toolbox drawer, drag out a couple of 'show leds' blocks from the Basic Toolbox drawer to the coding Workspace. Since we'll be using them as a guide, don't connect them to the 'on start' block. Remember, the blocks will be grayed out, but you can still create a smiling face in one and non-smiling face in the other by selecting the squares you want.

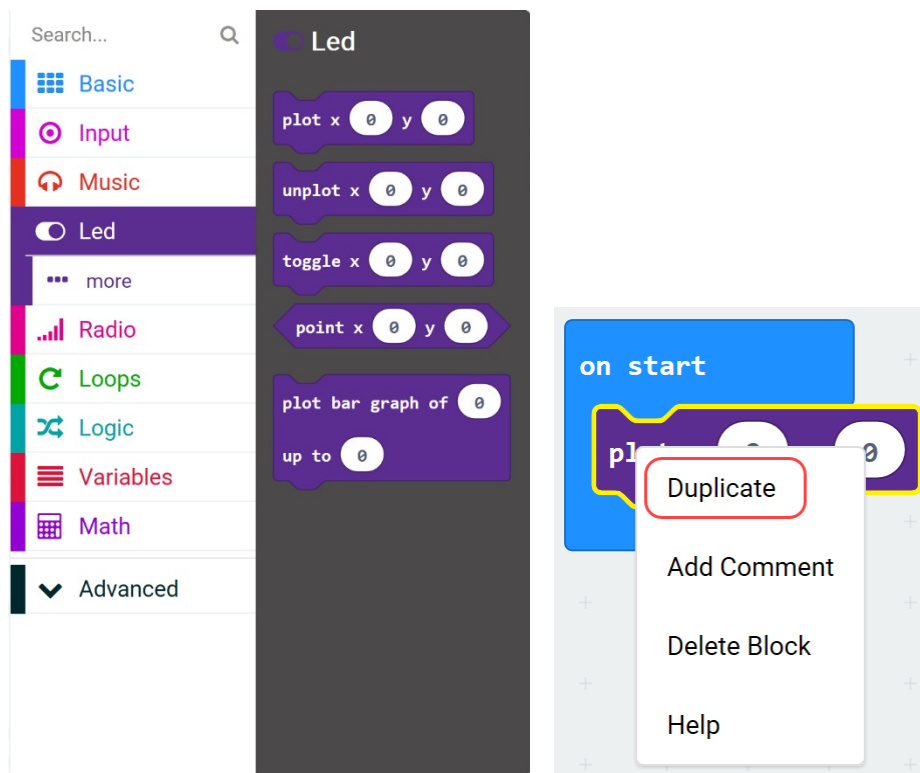




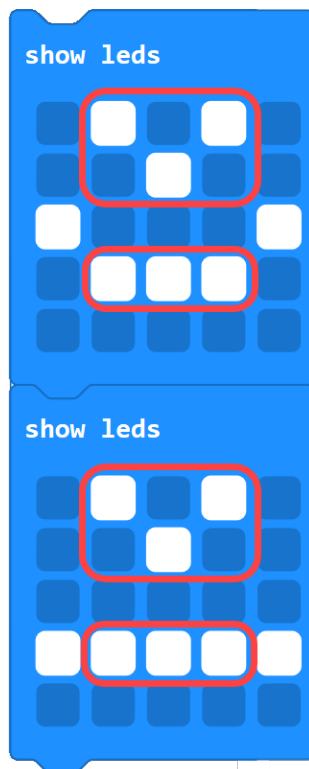
Plot coordinates for LEDs that are in both images

3. From the LED Toolbox drawer, drag out six 'plot x y' blocks and connect them inside the 'on start' block.

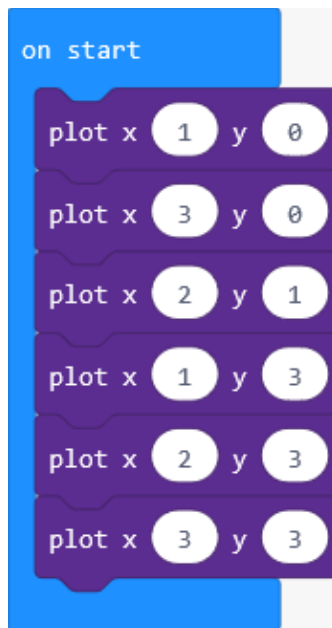
Tip: You can also right-click a block and select Duplicate to copy blocks.



4. Compare the two face images to determine which LEDs are on in both images.



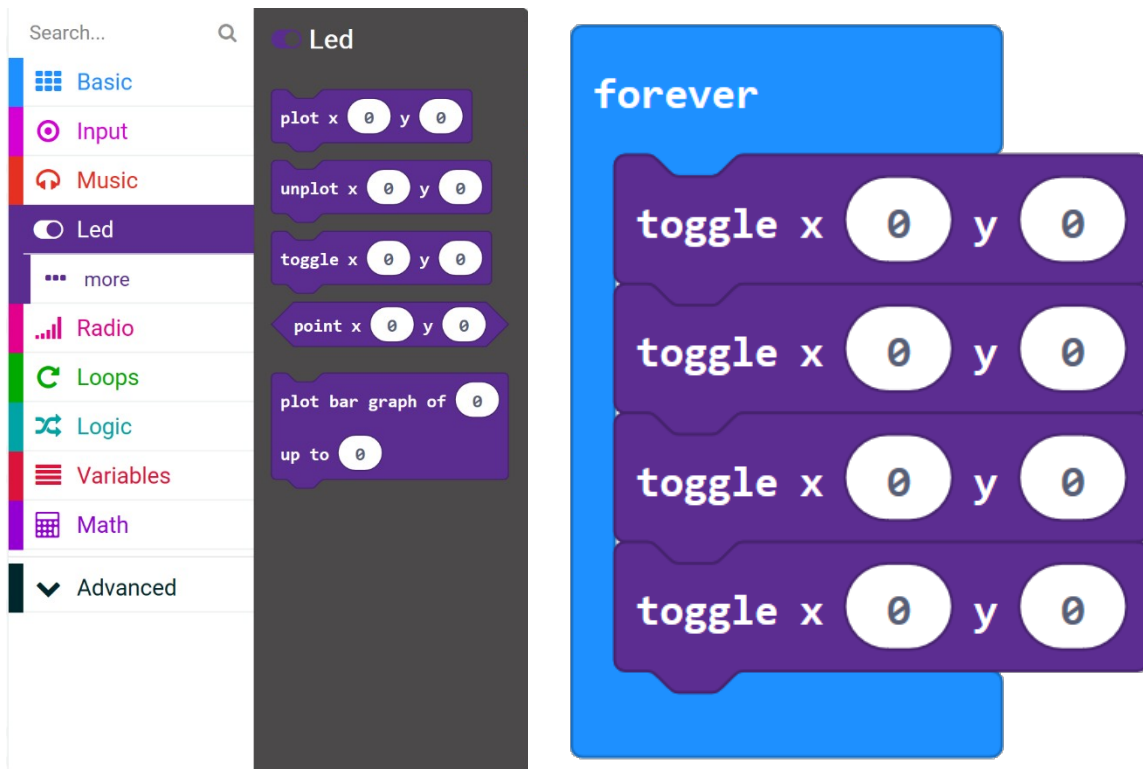
5. Update the default (0,0) coordinates with the correct (x,y) coordinates for the shared LEDs in both faces.



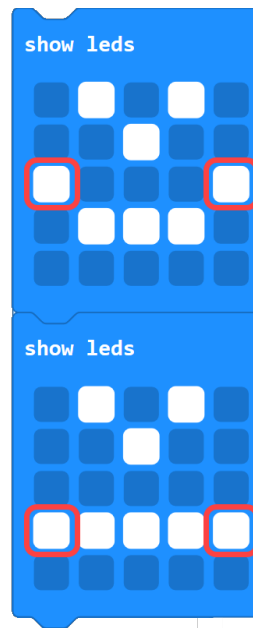
Code the toggle coordinates

Now, you can code for the four LEDs that change back and forth (on and off) as we switch from one face to the other and back again, over and over.

- From the LED Toolbox drawer, drag out four 'toggle x y' blocks and connect them into the 'forever' block. The 'toggle x y' block will change the status of an LED from on to off and off to on.



- Determine the coordinates for the LEDs that will toggle.



8. First, update the default (0,0) coordinates with the correct (x,y) coordinates for the two LEDs to create the smile in the first two 'toggle x y' blocks. Then, update the other two 'toggle x y' blocks with the correct (x,y) coordinates for the two LEDs to create the non-smile.



9. Run the code in the Simulator to check it out. It toggles all four blocks at the same time! Let's fix it.

Add a pause between the smile and non-smile

10. Let's slow it down a bit by placing a 'pause' block from the Basic Toolbox between the two pairs of 'toggle x y' blocks. Then, set the pause value to 250 milliseconds.

Mods

- Add a third image to the animation, perhaps a frown face.
- Make your own custom animation! What LEDs stay the same and which need to be toggled?

## Coding activity 2: Random patterns generator

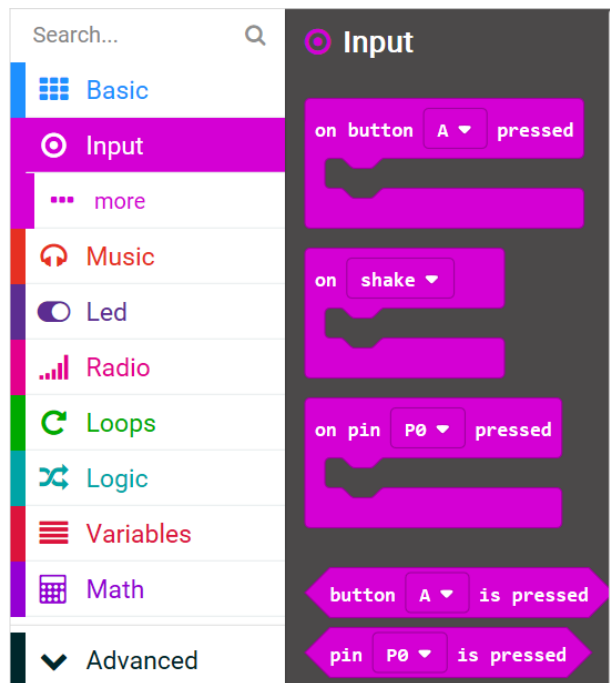
A short exercise using a loop to generate random LED patterns and then check the status of a specific LED.

## Pseudocode

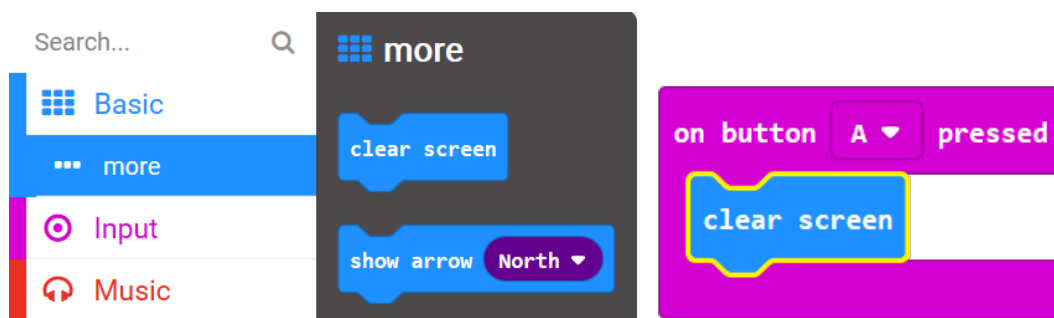
- On button A pressed, we'll use a loop to turn on a random set of LED lights on our micro:bit.
- Our display will have one LED lit for each column (or x-coordinate value) from 0 through 4.

## Code button A

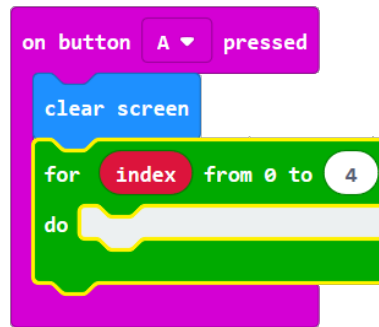
1. Start a new MakeCode project and name it. We don't need the 'on start' or 'forever' blocks in this activity, so delete them from the coding Workspace or move them to the side.
2. From the Input Toolbox drawer, select the 'on button pressed' block.



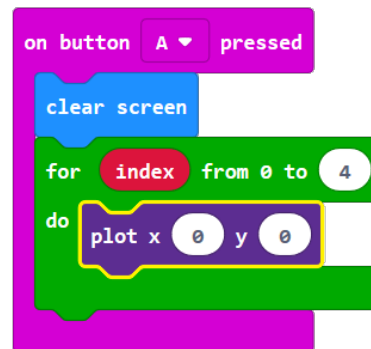
3. From the Basic – More Toolbox drawer, drop in a 'clear screen' block.



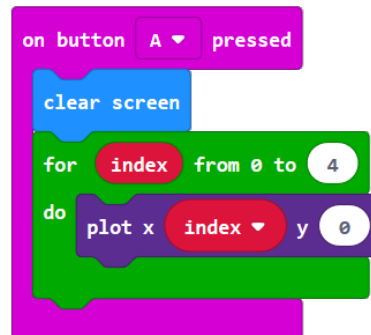
4. From the Loops Toolbox drawer, drop in a 'for' block under the 'clear screen' block.



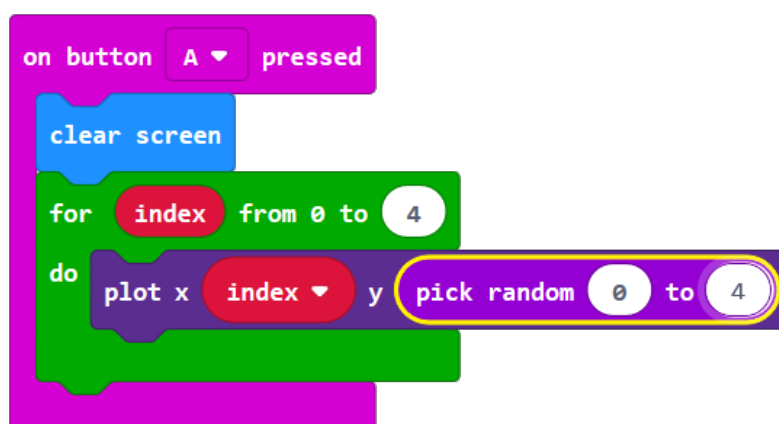
5. From the LED Toolbox drawer, drop a 'plot x y' block into the 'for' block.



6. From the Variable Toolbox drawer, drag an 'index' variable into the x value in the 'plot x y' block.



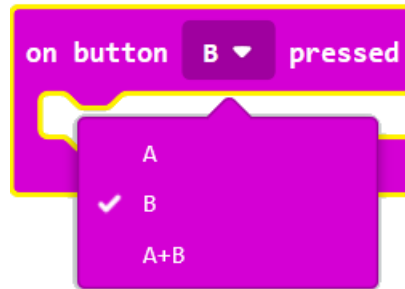
7. From the Math Toolbox drawer, drag a 'pick random' block into the y value and change the second parameter from 0 to 4.



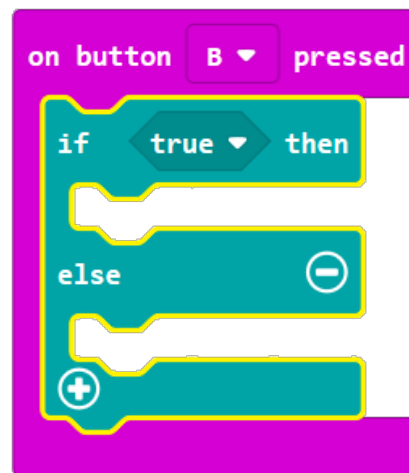
## Code button B

Now, you'll code to check the on/off state of one of the LEDs with button B.

- From the Input Toolbox drawer, drag an 'on button pressed' block to the coding Workspace and use the dropdown menu to select B.

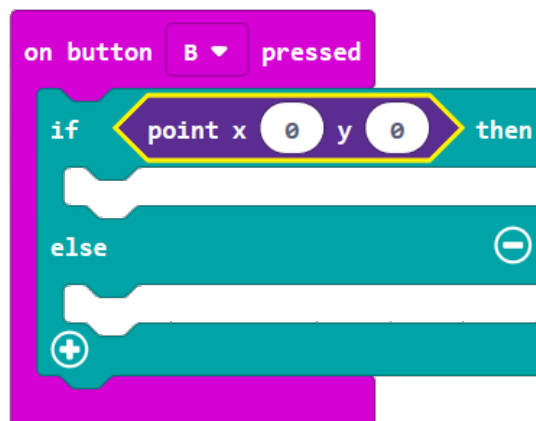


- On button B pressed, you'll use an 'If then else' block from the Logic Toolbox drawer.

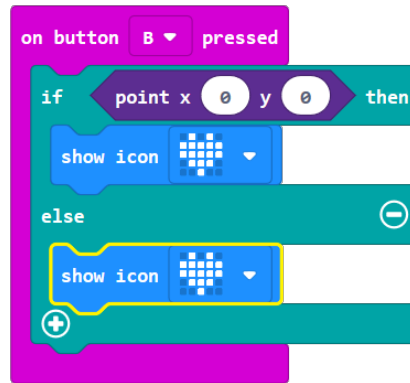


- From the LED Toolbox drawer, drop a 'point x y' block into the 'if' condition to check the current on/off state of a specific LED. This means:

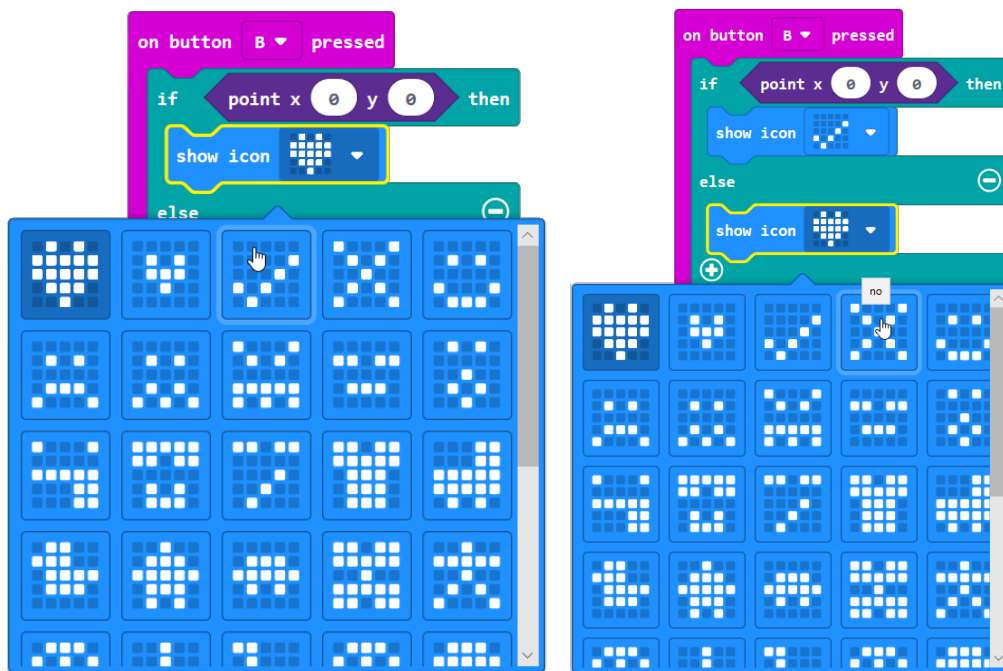
- If the LED located at (0,0) on the micro:bit is currently on, the 'point x y' block will return true.
- If the LED located at (0,0) is currently off, the 'point x y' block will return false.



11. For this exercise, you'll use the two built-in icons to display the LED's current status. From the Basic Toolbox drawer, drag two 'show icon' blocks into each of the 'then' and 'else' clauses.



12. Use the drop-down menu to select the check mark for Yes, and the X icon for No.



Try it out!

- Download the program to your micro:bit.
- Press button A to create a random pattern.
- Press button B to check and display the status of the specific LED.

## Mods

Add a loop to test for all coordinates on the micro:bit when button B is pressed instead of testing for the (0, 0) LED only.

## Coding activity 3: Brightness

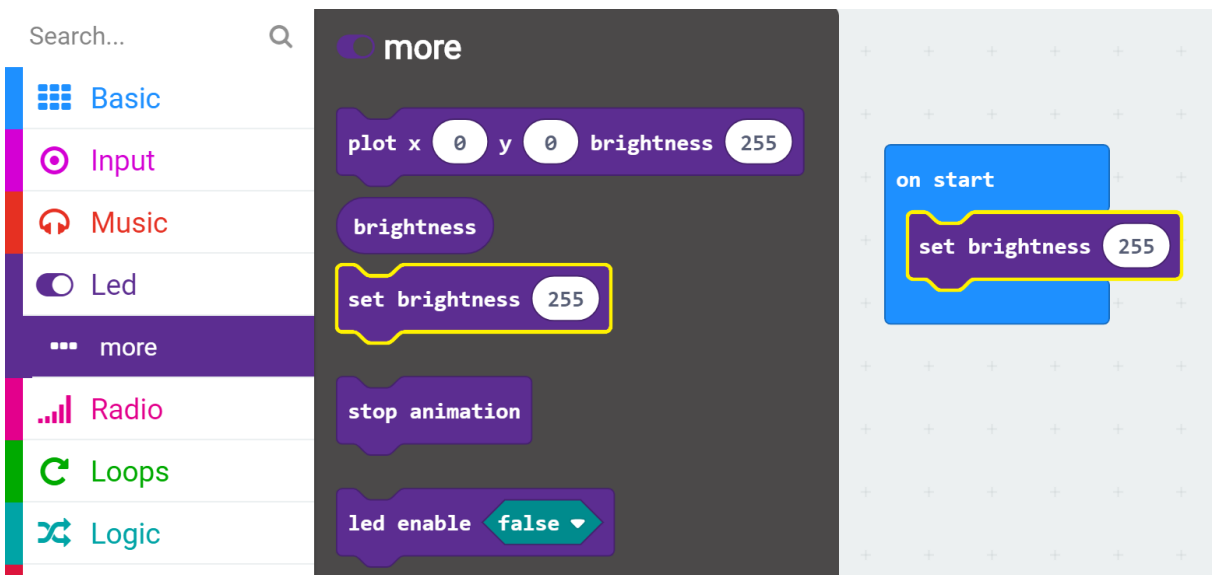
A short exercise in using the brightness settings for the micro:bit LEDs. Our program will change the brightness of the LED's and numerically display the brightness level.

### Pseudocode

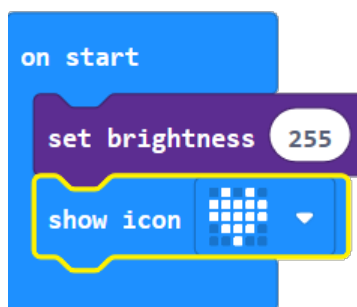
- You'll set the brightness level for the LEDs to the highest level on start and then use on button A pressed to decrease the brightness level, and on button B pressed to increase the brightness level.
- You'll use on button A+B pressed to check and display numerically the current brightness level.

### Code for the start

1. Start a new project in MakeCode and name it.
2. Select the LED Toolbox drawer. You'll notice a "...more" Toolbox drawer appear; select that and drag a 'set brightness' block onto the coding Workspace. Drop this into the 'on start' block.



3. From the Basic Toolbox drawer, drag a 'show icon' block and drop it after the 'set brightness' block so we will have an image to look at.

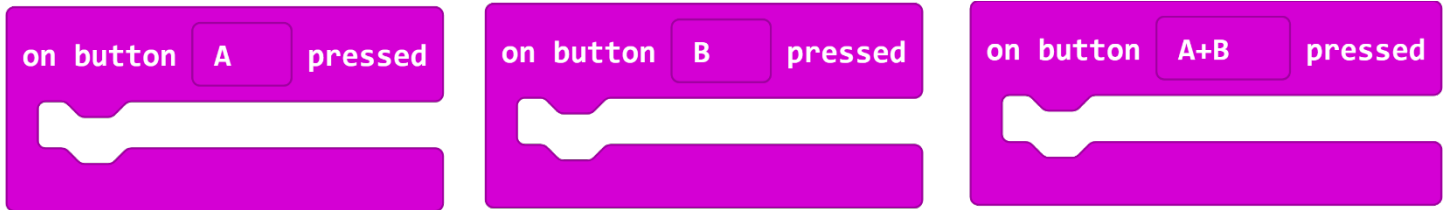


### Code for button A and button B

4. From the Input Toolbox drawer, drag out three 'on button pressed' blocks onto your coding Workspace.



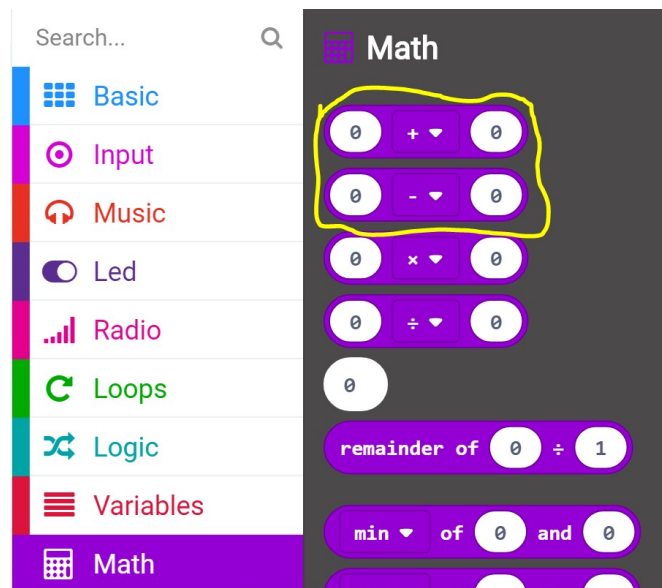
5. Leave one 'on button pressed' block with the default setting of A and use the dropdown menu to change the other to B, and the third to A+B (this is when buttons A and B are pressed together)



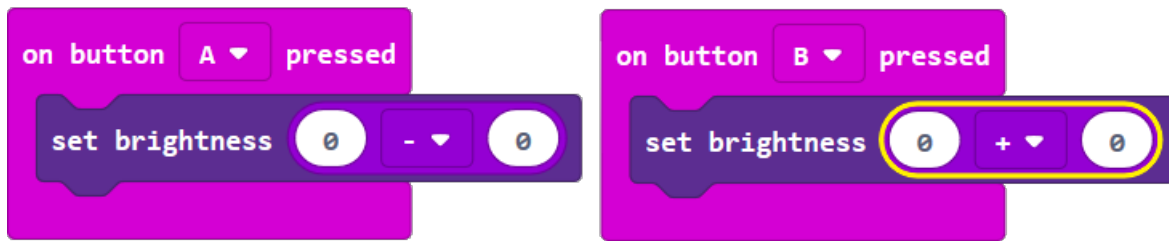
6. From the LED ...more Toolbox drawer, drag out two more 'set brightness' blocks and drop one each into the 'on button A pressed' and 'on button B pressed' blocks.



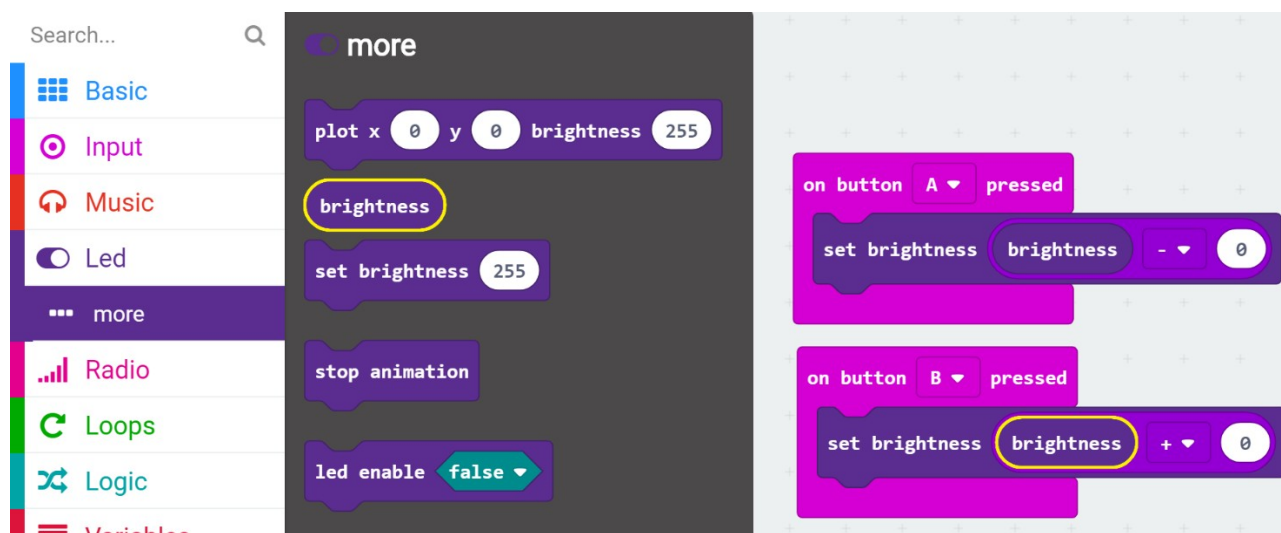
7. From the Math Toolbox drawer, drag out an addition block and a subtraction block.



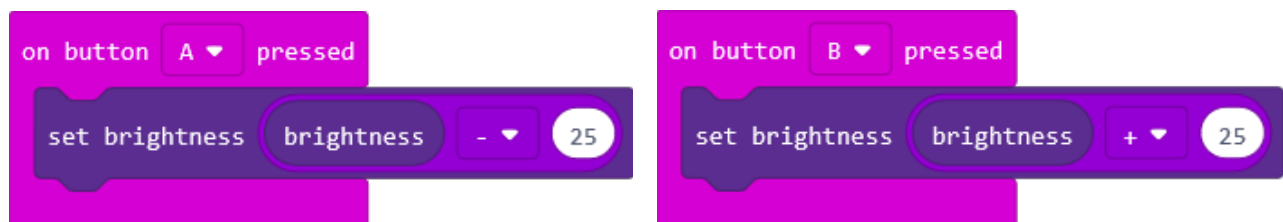
8. Drop the addition block within the 'set brightness' block in the 'on button B pressed' block, and the subtraction block within the 'set brightness' block in the 'on button A pressed' block, replacing the default value of '255'.



9. From the LED ...more Toolbox drawer, drag out two 'brightness' value blocks to the Workspace and drop one into the first slot of the subtraction block, and drop the second one into the first slot of the addition block.



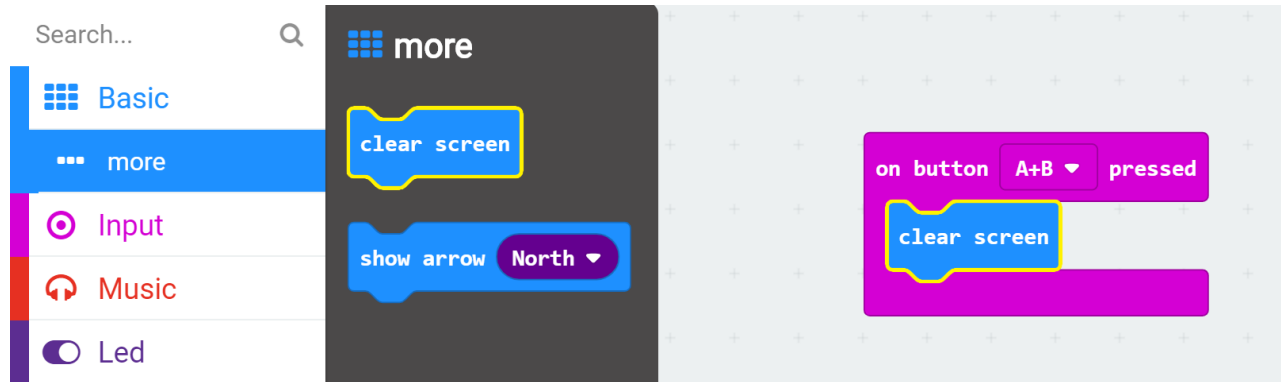
10. Change the default value of 0 on the right side of each math expression to 25.



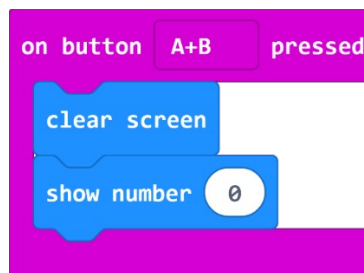
## Code for button A + B together

Since you can't see if our program is working in the simulator, let's add a check into our code.

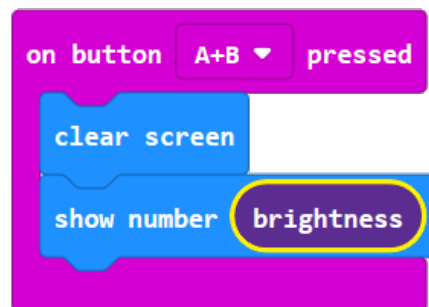
11. From the Basic ...more Toolbox drawer, drag a 'clear screen' block onto the Workspace and drop it into the 'on button A+B pressed' block.



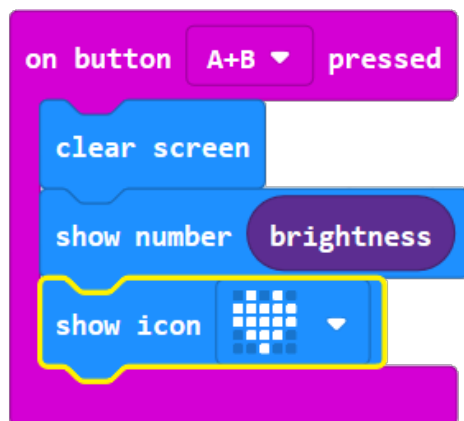
12. From the Basic Toolbox drawer, drag a 'show number' block onto the Workspace and drop it below the 'clear screen' block.



13. From the LED ...more Toolbox drawer, drag another 'brightness' value block onto the Workspace and drop it into the 'show number' block, replacing the default 0 value. This will clear the screen when buttons A and B are pressed and display the current brightness level as a number on the micro:bit screen.



14. Now, from the Basic Toolbox drawer, drag a 'show icon' block to the coding Workspace and connect it below the 'show number' block. This will re-display the image we used on start of the program.



Try it out!

Check it in the simulator first, then download the program to the micro:bit to run the program.

Mod

What happens if adding 25, or subtracting 25, from the current brightness level would result in a sum or difference outside of the 0 to 255 brightness range?

# Lesson C: Screensaver or game project

---

## Activity: Make a screensaver or game project

Use what you now know about LEDs, coordinates, and brightness to create your own project: a screensaver or a game. You should find a way to use coordinates in your program. Even better, use variables to store and update your coordinates.

Your task is to create:

- A “screen saver” animation using the plot/unplot blocks. You can fill the screen line by line, pausing between each one, or fill it with a random constellation of stars.

– OR –

- A game that uses sprites to manage the x- and y-coordinate values of the different objects.

Your project might use variables to store the values of sprites, which are special structures that contain an x- and a y-coordinate together that describe the sprite’s location as one LED on the screen.

### Screensavers

One type of project is a screensaver. A long time ago, computers and televisions used cathode ray tube (CRT) screens for displays. The glass screen of the display was coated on the back with phosphor, a substance that glows when painted with electrons from an electron gun at the other end of the tube. When the same area of the screen was painted (or excited) over and over again by the stream of electrons, that part of the screen would sometimes “freeze” with the same image burned into the phosphor for good. This was called burn-in.



Normally, if a show was running or if someone was actively using the computer, the display changed often enough that burn-in wasn’t a problem. Programmers learned to create a demo screen with an animation that would run whenever the screen was idle. Today, nearly all computers and television sets use LCD displays, which are not affected by burn-in. But you can still find a screen saver in nearly every computer’s Settings panel as an opportunity to show off some neat graphics or animation.

## Project expectations

Follow the design thinking approach and make sure your project meets these specifications:

- Uses at least three of the different kinds of 'plot', 'unplot', 'toggle', and 'point x y' blocks, and uses variables to update the coordinates in some way
- Uses plotted LEDs in a meaningful way
- The program compiles and runs as intended and includes meaningful comments in code
- Provide the written Reflection Diary entry (which we'll talk about after you complete your project)

The design thinking process:

1. **Empathize** by learning more about your target audience.
2. **Define**: Understand and identify your audience's problems or needs.
3. **Ideate**: Brainstorm several possible creative solutions.
4. **Prototype**: Construct rough drafts or sketches of your ideas.
5. **Test** your prototype solutions and refine until you come up with the final idea.

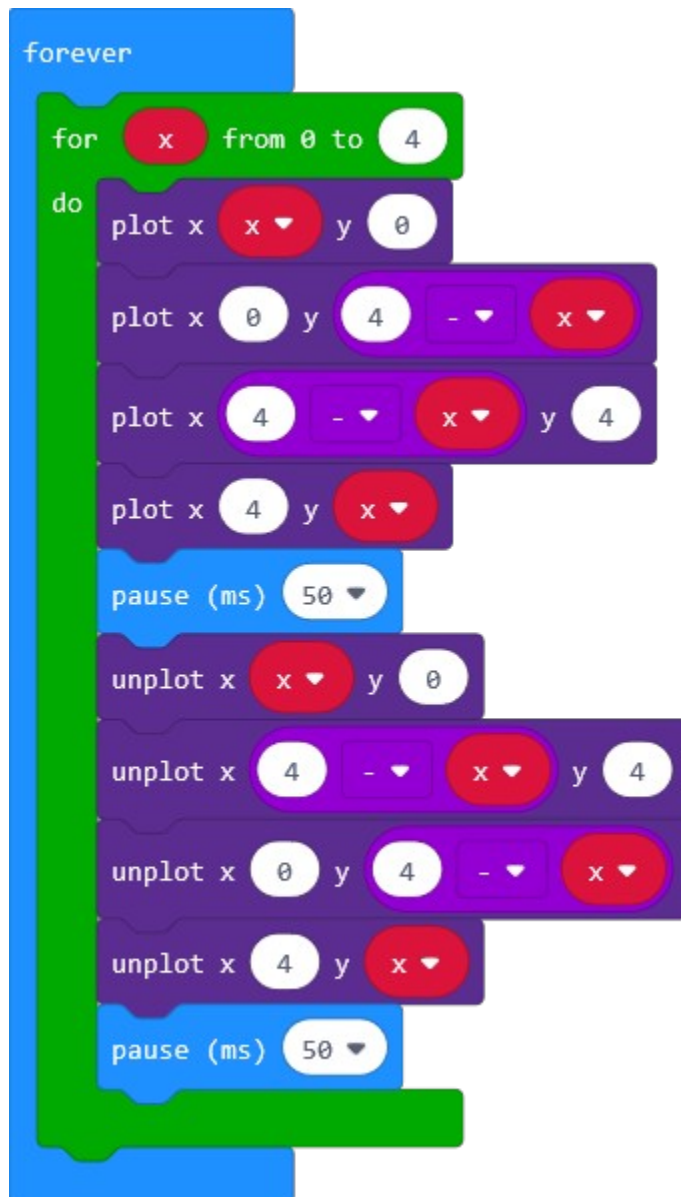
Use this space to track your design thinking process:

## Project examples

### 1. Fireworks screensaver

This project uses a for loop with the plot/unplot blocks to create a symmetrical design on the screen. The following sample code used a subtraction operation to get a variable that decreases as the index variable in the loop increases.

Complete code

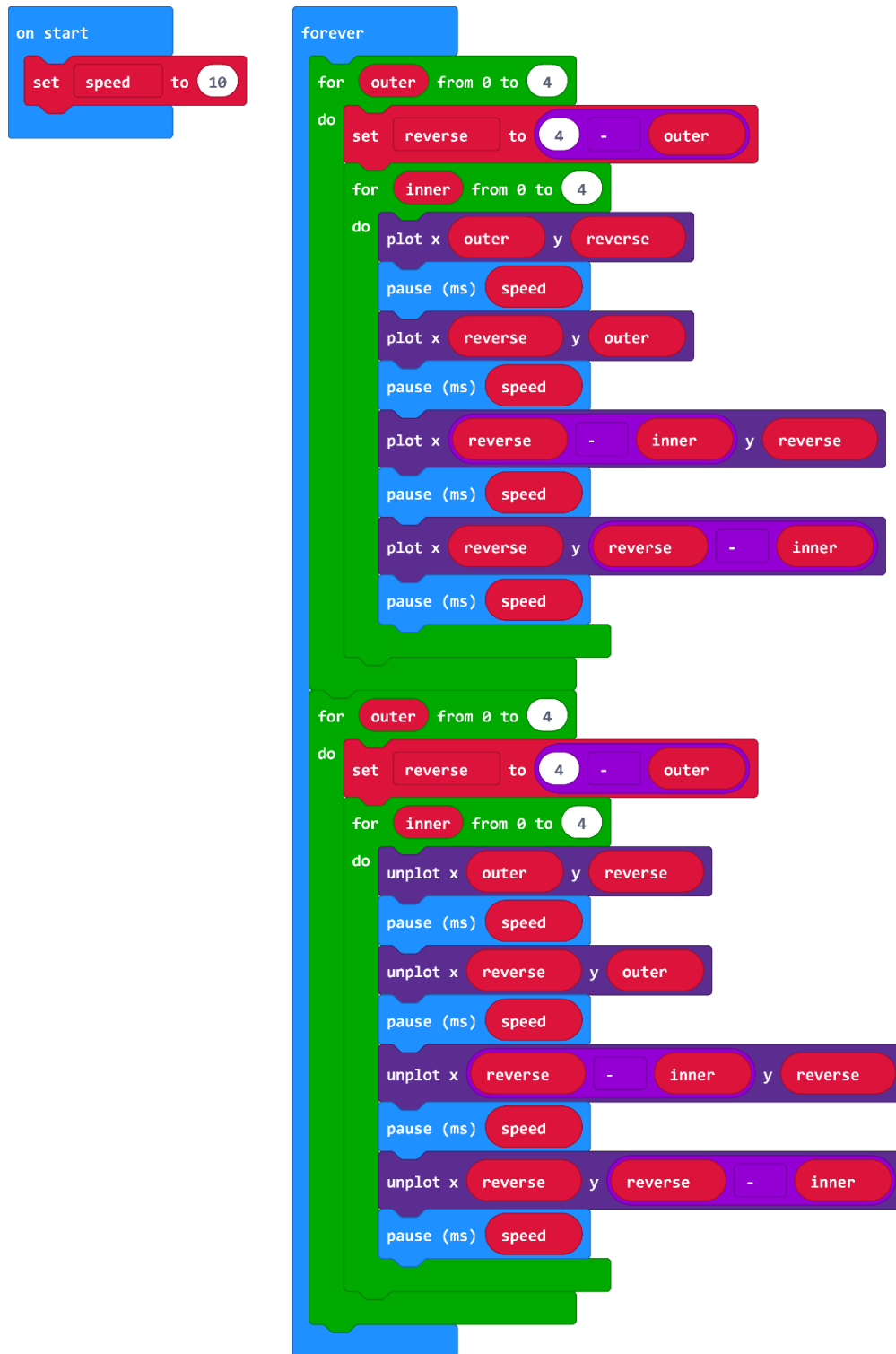


Solution link: [makecode.microbit.org/\\_97A6Ru6LELcP](https://makecode.microbit.org/_97A6Ru6LELcP)

## 2. Cascade screensaver

This example creates a diagonal cascading effect across the screen. Note the use of a variable (speed) to allow you to easily change the speed of the animation by changing just one number value.

Complete code



Solution link: [makecode.microbit.org/\\_Y2TU9cgWz07m](https://makecode.microbit.org/_Y2TU9cgWz07m)



### 3. Dodge ball game

This is a Dodge Ball game that uses one sprite (dodger) to try to avoid another sprite (ball). You use the A and B buttons to move the dodger to avoid the balls that are falling from the top of the screen.

#### Complete code

```
on start
  set ball to create sprite at x: pick random 0 to 5 y: 0
  set dodger to create sprite at x: 2 y: 4
  set score 0

forever
  if dodger touching ball ? then
    game over
  else if ball y < 4 then
    ball change y by 1
    pause (ms) 250
  else
    change score by 1
    ball set y to 0
    ball set x to pick random 0 to 5

on button A pressed
  if dodger x > 0 then
    dodger change x by -1

on button B pressed
  if dodger x < 4 then
    dodger change x by 1
```

Solution link: [makecode.microbit.org/\\_E9b733huX2DP](https://makecode.microbit.org/_E9b733huX2DP)

## Project scoring rubric

| Assessment elements  | 1                                                               | 2                                                                                                       | 3                                                                                                        | 4                                                                                                                                                                                                                      |
|----------------------|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Coordinates and LEDs | No plot/unplot/toggle/point x y blocks are implemented.         | At least one of the different kinds of plot/unplot/toggle/point x y blocks is used in a meaningful way. | At least two of the different kinds of plot/unplot/toggle/point x y blocks are used in a meaningful way. | Uses at least 3 of the different kinds of plot/unplot/toggle/point x y blocks in a meaningful way, and uses variables to update coordinates.                                                                           |
| micro:bit program    | micro:bit program lacks three or more of the required elements. | micro:bit program lacks two of the required elements.                                                   | micro:bit program lacks one of the required elements.                                                    | micro:bit program: <ul style="list-style-type: none"> <li>■ Uses plotted LEDs in a way that is integral to the program</li> <li>■ Compiles and runs as intended</li> <li>■ Uses meaningful comments in code</li> </ul> |

# Reflection Diary

## Expectations

Write a reflection of about 150–300 words addressing the following points:

- Did you create a screensaver? A game? Something different? How did you decide?
- If you created a game, what is the object of the game?
- How does your project use coordinates?
- Describe something in your project that you are proud of.
- Describe a difficult point in the process of designing this program and explain how you resolved it.
- What feedback did your beta testers give you? How did that help you improve your design?
- Publish your MakeCode program and include the link.

## Diary entry scoring rubric

| Assessment elements | 1                                                              | 2                                                    | 3                                                    | 4                                   |
|---------------------|----------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|-------------------------------------|
| Diary entry         | Diary entry is missing three or more of the required elements. | Diary entry is missing two of the required elements. | Diary entry is missing one of the required elements. | Diary entry addresses all elements. |

# Glossary of key terms

---

**Coordinates:** A coordinate represents a position or location. Coordinates tell a computer program where an action should take place by providing the location for the action. On the micro:bit, the coordinates are represented on a 5 x 5 LED grid. Also see coordinate pair, x-coordinate, and y-coordinate.

**Coordinate pair:** The combination of the x- and y-coordinates represented as (x,y).

**x-coordinate:** On the micro:bit, the x-coordinate is a measure of how many units a point is horizontally from the origin and increases in value from left to right on the 5 x 5 LED grid.

**y-coordinate:** On the micro:bit, the y-coordinate is a measure of how many units a point is vertically from the origin and increases in value from top to bottom on the 5 x 5 LED grid.